

Faggruppemøte Databasedykkerne

Mars 2026

Hva gjør man når man fucker opp?

Hva gjør man når man har tenkt å fucke opp?

Hvordan kan ting rulles tilbake?

Snapshot og backup

Begrepene brukes på to nivåer:

Infrastruktur — disk eller filsystem tar bildet:

- Databasen vet ikke at det skjer
- Databasen kan ha data i minnet

Database:

- Database-funksjonalitet som garanterer gyldig tilstand

Snapshot og Restore Points

- Et fryst bilde av databasen på ett tidspunkt
- Avhengig av at den underliggende databasen er intakt
- Lagrer "diff-filer" — vokser etter hvert som originalen endres
- Ingenting med transaksjonsloggen å gjøre
- Kan spørres mot direkte — uten å restore først

MSSQL — Database Snapshot

```
-- 1. Ta et snapshot
CREATE DATABASE demo_snap ON (
    NAME = demo_db, FILENAME = '/var/opt/mssql/snapshots/demo_snap.ss'
) AS SNAPSHOT OF demo_db

-- 2. Spørre direkte mot snapshot (read-only)
SELECT * FROM demo_snap.dbo.orders WHERE order_date > '2026-03-01'

-- Sammenlign snapshot mot live
SELECT snap.amount, live.amount
FROM demo_snap.dbo.orders AS snap
JOIN demo_db.dbo.orders AS live ON snap.id = live.id
WHERE snap.amount <> live.amount

-- 3. Restore ved behov
RESTORE DATABASE demo_db FROM DATABASE_SNAPSHOT = 'demo_snap'
```

Oracle — Restore Point

```
-- 1. Ta et restore point
CREATE RESTORE POINT before_change GUARANTEE FLASHBACK DATABASE;

-- 2. Spørre direkte mot historiske data (AS OF, bruker undo)
SELECT * FROM orders
AS OF TIMESTAMP TO_TIMESTAMP('2026-03-19 10:00:00', 'YYYY-MM-DD HH24:MI:SS')
WHERE order_date > DATE '2026-03-01';

-- Eller relativt i tid
SELECT * FROM orders
AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '30' MINUTE);

-- 3. Flashback ved behov
FLASHBACK DATABASE TO RESTORE POINT before_change;
```


PostgreSQL

- Har `pg_create_restore_point()`, men det fungerer litt annerledes

Backup

En selvstendig kopi av databasen på ett tidspunkt.

- Kan ikke spørres mot — må restores til en kjørende database først
- Uavhengig av den opprinnelige databasen — fungerer selv om originalen er tapt
- Bringer deg tilbake til nøyaktig det tidspunktet backupen ble tatt

Database	Verktøy
MSSQL	<code>BACKUP DATABASE demo TO DISK = '/backup/demo.bak'</code>
PostgreSQL	<code>pg_basebackup</code>
Oracle	<code>RMAN BACKUP DATABASE PLUS ARCHIVELOG</code>

Backup + transaksjonslogg = ❤️

Transaksjoner

En transaksjon er en gruppe operasjoner som enten alle går gjennom – eller ingen.

```
BEGIN TRANSACTION
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1
```

```
UPDATE accounts SET balance = balance + 100 WHERE id = 2
```

```
COMMIT          -- alt gikk bra
```

```
-- eller:
```

```
ROLLBACK        -- angre alt siden BEGIN
```

Isolation

Nivå	Hva du kan se
READ UNCOMMITTED	Andres ucommittede endringer (dirty read)
READ COMMITTED	Bare committede data (<i>standard i MSSQL</i>)
REPEATABLE READ	Samme rad gir samme svar gjennom hele transaksjonen
SERIALIZABLE	Fullstendig isolasjon – som om du er alene i databasen

Høyere isolasjon er "tryggere", men mer låsing og lavere ytelse.

Obs!

Mange database-verktøy har en **auto-commit**-knapp.

IDE	Auto-commit-toggle	Standard
SSMS	Nei	Auto-commit ON
Azure Data Studio	Nei	Auto-commit ON
DataGrip	Ja (verktøylinje)	Auto-commit ON
DBeaver	Ja (verktøylinje)	Auto-commit ON
Oracle SQL Developer	Ja (verktøylinje)	Auto-commit OFF

I SSMS og Azure Data Studio: Må skrive `BEGIN TRANSACTION` selv.

En åpen transaksjon låser rader. Husk å `COMMIT` eller `ROLLBACK` når du er ferdig.

Data Definition Language (DDL) er ikke alltid transaksjonelt

I **MSSQL** og **PostgreSQL** er DDL transaksjonelt — **DROP TABLE** kan rulles tilbake:

```
-- MSSQL / PostgreSQL
BEGIN TRANSACTION
DROP TABLE orders
ROLLBACK                -- orders er tilbake
```

I **Oracle** committer DDL implisitt — **ROLLBACK** hjelper ikke:

```
-- Oracle
DROP TABLE orders;    -- implisitt COMMIT før og etter
ROLLBACK;              -- orders er borte
```

Transaksjonsloggen

Loggen brukes til crash recovery, rollback og replikering.

Kjært barn har mange navn:

Database	Navn
MSSQL	Transaction Log
PostgreSQL	WAL
Oracle	Redo Log

Checkpoint

Databasen skriver jevnlig endringer fra minnet til disk. Etter dette trengs ikke de tilhørende loggpostene lenger for å overleve et krasj.

Loggene kan da enten:

- **Slettes**
- **Beholdes**

Dette styres av recovery-oppsettet.

PostgreSQL: WAL

- *Write-Ahead Log* — ligger i `pg_wal/`
- Segmentfiler på typisk 16 MB hver
- Uten arkivering: segmenter gjenbrukes/slettes etter checkpoint
- Med arkivering: aktiveres med `archive_mode = on` + `archive_command`

```
# postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'cp %p /backup/wal/%f'
```

Med arkivering bevarer segmentene til `archive_command` har kjørt.

MSSQL: Transaction Log

Én transaksjonslogg per database — en `.ldf` -fil som ligger i `/var/opt/mssql/data/`.

Det viktigste å konfigurere er **Recovery Model**:

Recovery Model	Loggen etter checkpoint	Point-in-time recovery?
<code>SIMPLE</code>	Slettes automatisk	Nei
<code>FULL</code>	Bevares til logg-backup kjøres	Ja

```
ALTER DATABASE demo SET RECOVERY FULL
```

PostgreSQL: WAL

WAL (*Write-Ahead Log*) — samme konsept som MSSQL Transaction Log.

Segmentfiler lagres i `pg_wal/`.

Konfigureres i `postgresql.conf` :

Oppsett	Loggen etter checkpoint	Point-in-time recovery?
Uten arkivering	Slettes automatisk	Nei
Med arkivering	Bevares til <code>archive_command</code> har kjørt	Ja

```
wal_level = replica
archive_mode = on
archive_command = 'cp %p /backup/wal/%f'
```

Oracle: Logger

Redo Logs er det samme konseptet som WAL og Transaction Log — loggfiler som kan arkiveres.

I tillegg har Oracle to unike mekanismer:

Logger	Hva den lagrer	Brukes til
Redo Logs	Alle endringer som har skjedd	Point-in-time recovery (som WAL/Transaction Log)
Undo	Hva som fantes <i>før</i> en endring	Rollback, Flashback Query, Flashback Table
Flashback Logs	Blokk-snapshots bakover i tid	Flashback Database — hele databasen tilbake i tid

Point-in-time recovery

Gjenopprette databasen til et *nøyaktig tidspunkt* — ikke bare til siste backup.

Krever to ting satt opp **på forhånd**:

1. En **full backup** tatt jevnlig
2. En **ubrutt kjede av loggfiler** fra backup-tidspunktet og frem til ønsket tidspunkt

Mangler én loggfil i kjeden → kan ikke komme lenger enn til hullet.

Restore til akkurat det sekundet du vil

Krever to ting satt opp **på forhånd**:

1. **Full backup** tatt jevnlig
2. **Ubrutt kjede av loggfiler** fra backup-tidspunktet frem til ønsket tidspunkt

Mangler én loggfil i kjeden → kan ikke komme lenger enn til hullet.

```
-- MSSQL: restore database, deretter spol frem i loggen
RESTORE DATABASE demo
  FROM DISK = '/backup/demo_full.bak'
  WITH NORECOVERY

RESTORE LOG demo
  FROM DISK = '/backup/demo_log.bak'
  WITH STOPAT = '2026-03-18T14:32:00', RECOVERY
```

Hva er mulig?

To tilnærminger til å angre en feil:

	Restore til et tidspunkt	Selektiv undo
Hva det er	Spol hele databasen tilbake	Angre én operasjon, resten urørt
Krever	Full backup + ubrutt loggkjede	Undo-data eller flashback-logger
MSSQL	Ja (FULL recovery model)	Nei — ikke innebygd
PostgreSQL	Ja (WAL-arkivering)	Nei — ikke innebygd
Oracle	Ja (ARCHIVELOG-modus)	Ja — Flashback

Oracle er eneste database av de tre med innebygd støtte for selektiv undo.

Oracle

Oracle har tre separate mekanismer som jobber parallelt:

Mekanisme	Hva den lagrer	Retning	Typisk bruksområde
Redo Log	Hva som <i>skjedde</i>	Fremover	Crash recovery, PITR, replikering
Undo	Hva som <i>var der før</i>	Bakover	Rollback, read consistency, kortvarig historikk
Flashback logs	Blokk-snapshots bakover	Bakover	Flashback Database — hele databasen tilbake i tid

Disse er uavhengige av hverandre. Flashback Query og Flashback Table bruker **undo**. Flashback Database bruker **flashback logs**.

Oracle Flashback

Verktøy	Hva den gjør	Mekanisme	Rekkevidde
Flashback Query	Les historiske data med AS OF	Undo	Timer (UNDO_RETENTION)
Flashback Table	Rull tilbake én tabell	Undo	Timer (UNDO_RETENTION)
Flashback Drop	Hent tilbake en droppet tabell	Recycle Bin	Til tabellen slettes permanent
Flashback Database	Rull tilbake hele databasen	Flashback logs	Dager (konfigurerbart)

```
-- Flashback Table: angre endringer på én tabell  
FLASHBACK TABLE orders TO TIMESTAMP (SYSTIMESTAMP - INTERVAL '30' MINUTE);  
  
-- Flashback Drop: hent tilbake tabell som ble droppet  
FLASHBACK TABLE orders TO BEFORE DROP;
```

Hva gjør man for å unngå å fucke opp?

Send meg en DM med noe du gjør for å dra ned risikoen.

Tips og triks Databasedykkerne edition

Bruk transaksjoner!

1. Lag transaction
2. Gjør endring
3. Les ut resultat
4. Commit 

Sjekk hvor mange rader som er påvirket:

```
SELECT @@ROWCOUNT;
```

Skriv WHERE clause først

Test den med SELECT før DELETE.
Se hvor mange rader som er påvirket.

```
SELECT * FROM Customers WHERE CustomerID = 1;
```

```
DELETE FROM FROM Customers WHERE CustomerID = 1;
```

Ikke tilgang til å skrive i Prod

Må utføres av andre (les plattform)

Bruker med begrensa rettigheter

Ikke DBO!

Mindre å fucke opp

Postgres dump-filer lokalt 🍷

Kjør akkurat det samme i Test først 🧐

Lag en backup-tabell

```
CREATE TABLE MY_TABLE_BACKUP_2025-05-01
```

```
SELECT *
```

```
INTO #backup_users
```

```
FROM Users
```

```
WHERE LastLogin < '2023-01-01';
```

```
SELECT *
```

```
INTO Users_BACKUP_20260319
```

```
FROM Users
```

```
WHERE LastLogin < '2023-01-01';
```

Konsultere våre venner 🤖

Språkmodell eller menneske

Ikke tukle med ting tidlig om morran eller etter to-knekken ☕